# Guía paso a paso — Clase 5: Redes multicapa y retropropagación

**Temas:** capas ocultas · pesos como matrices · pase hacia adelante · retropropagación · entrenar una red multicapa · ¡XOR resuelto!
**Duración:** ~2 h 30 min
**Material:** diapositivas `Clase5_Diapositivas` (PDF y PowerPoint, 21 diapositivas) · repositorio https://github.com/datahuba/curso-redes-neuronales
**Requisito:** Clases 3 y 4 (sigmoide, descenso por gradiente, regla delta)
**Lecturas base** (libro del Dr. Irving Vasquez):

- Redes neuronales multicapa
- Retropropagación

## Objetivos de la clase

Al terminar, cada alumno podrá:

1. Explicar por qué una capa oculta permite resolver problemas como XOR.
2. Calcular el pase hacia adelante de una red de dos capas con matrices, cuidando los tamaños.
3. Aplicar a mano un paso de la **retropropagación**.
4. Entrenar una red multicapa en Python y leer su curva de aprendizaje.

La guía tiene cuatro partes:

- **Parte A:** preparar las computadoras antes de clase.
- **Parte B:** teoría explicada desde cero, con ejemplos resueltos.
- **Parte C:** práctica en la computadora.
- **Parte D:** problemas frecuentes.

---

## Agenda

| Tiempo | Tema | Diapositivas |
|---|---|---|
| 0:00 | Presentación, objetivos y repaso de la Clase 4 (10 min) | 1–4 |
| 0:10 | B1. ¿Por qué capas ocultas? (15 min) | 5–6 |
| 0:25 | B2. Pesos como matrices y pase hacia adelante (20 min) | 7–9 |
| 0:45 | C1. Práctica: notebook `1_09` (20 min) | 10 |
| 1:05 | Descanso (10 min) | 11 |
| 1:15 | B3. Retropropagación (30 min) | 12–16 |
| 1:45 | C2. Práctica: notebook `1_10` (15 min) | 17 |

| Tiempo | Tema | Diapositivas |
|---|---|---|
| 2:00 | C3. Entrenar la red completa y resolver XOR (20 min) | 18–19 |
| 2:20 | Quiz y cierre (10 min) | 20–21 |

# PARTE A — Antes de clase (15 min)

En **Anaconda PowerShell Prompt**:

```
cd ~\projects\curso-redes-neuronales
```

```
git pull
```

```
conda activate nni
```

```
python docs/practica_05.py
```

**Verificación:** la red aprende XOR. Deben salir las cuatro predicciones `0, 1, 1, 0`, la pérdida bajando de ≈ 0.26 a ≈ 0.0006, y al final `¡La red multicapa resuelve XOR!`.

Para ver la curva y la frontera que aprendió la red:

```
python docs/practica_05.py --graficar
```

**Checklist:**

- [ ] `python docs/practica_05.py` funciona.
- [ ] VS Code abre `notebooks/1_09_red_multicapa_ejercicio.ipynb` con el kernel **nni**.
- [ ] Tienes a mano las guías de los notebooks `1_09`, `1_10` y `1_11` (en `docs/guias_notebooks/` o en `entregables/Guias_de_los_notebooks.pdf`).
- [ ] En el pizarrón: el plano con los 4 puntos de XOR. Lo usarás para dibujar dos rectas.

# PARTE B — Teoría desde cero

## Repaso de la Clase 4 (5 min) — diapositivas 1–4

- "¿Cuáles son los tres pasos de la regla delta?" → término de error, incremento y actualización.
- "¿Qué problema quedó sin resolver?" → **XOR**: una sola neurona solo traza una recta, y XOR no se separa con una recta.

Hoy lo resolvemos.

## B1. ¿Por qué capas ocultas? (15 min) — diapositivas 5–6

### Dos rectas en lugar de una (diapositiva 5)

En el plano de XOR, los puntos (0, 1) y (1, 0) valen 1; los puntos (0, 0) y (1, 1) valen 0. Dibuja **dos** rectas:

- la recta A deja **abajo** solo el punto (0, 0) (como un OR);
- la recta B deja **arriba** solo el punto (1, 1) (como un AND).

Los puntos que valen 1 están **entre** las dos rectas. Entonces:

1. una neurona calcula la recta A;
2. otra neurona calcula la recta B;
3. una **tercera** neurona combina ambas: "arriba de A **y** debajo de B".

Las dos primeras forman la **capa oculta**; la tercera es la **capa de salida**.

**Idea clave:** cada neurona oculta traza una recta. La salida combina esas rectas y puede formar regiones con forma de franja, de esquina o de cualquier figura.

### La arquitectura (diapositiva 6)

- **Capa de entrada:** los datos (no calcula nada).
- **Capa oculta:** neuronas cuya salida no vemos directamente. De ahí su nombre.
- **Capa de salida:** la predicción.
- Esta red se llama **perceptrón multicapa** (MLP). Si tiene muchas capas ocultas, hablamos de **aprendizaje profundo**.

## B2. Pesos como matrices y pase hacia adelante (20 min) — diapositivas 7–9

### Pesos como matrices (diapositiva 7)

Red del notebook `1_09`: **4 entradas → 3 ocultas → 2 salidas**.

- Entre la entrada y la oculta hay 4 × 3 = 12 pesos. Van en una matriz $\mathbf{W_1}$ de **4 filas × 3 columnas**: filas = entradas, columnas = neuronas de la capa siguiente.
- Entre la oculta y la salida: $\mathbf{W_2}$ de **3 × 2**.

**Regla de los tamaños:** (a × b)·(b × c) = (a × c). El número del medio debe coincidir. Por ejemplo, (1 × 4)·(4 × 3) = (1 × 3).

### Pase hacia adelante (diapositiva 8)

Es la Clase 4 dos veces:

| Paso | Fórmula | Tamaño |
|---|---|---|
| Capa oculta | $H_1 = X \cdot W_1 + B_1$ y $A_1 = \sigma(H_1)$ | 1 × 3 |
| Capa de salida | $H_2 = A_1 \cdot W_2 + B_2$ y $\hat{Y} = \sigma(H_2)$ | 1 × 2 |

**La salida de una capa es la entrada de la siguiente.**

### Contar parámetros (diapositiva 9)

- **Pesos:** 4·3 + 3·2 = 18.
- **Sesgos:** 3 + 2 = 5.
- **Total: 23 parámetros**.

Regla general: por cada capa, (entradas + 1) × neuronas. Aquí: (4 + 1)·3 + (3 + 1)·2 = 15 + 8 = 23.

---

## B3. Retropropagación (30 min) — diapositivas 12–16

### El problema (diapositiva 12)

Con una neurona sabíamos su error: y − ŷ. Pero **¿cuál es el error de una neurona oculta?** Nadie nos dice qué debió producir.

**La solución:** calcular el error en la salida y **mandarlo hacia atrás**. Cada neurona oculta recibe una parte proporcional al peso que la une con la salida: si influyó mucho, carga más culpa.

**Analogía:** un equipo pierde un partido. El entrenador reparte la responsabilidad según cuánto participó cada jugador en la jugada decisiva.

### Las fórmulas (diapositiva 13)

Con la sigmoide, f′ = salida·(1 − salida).

| Capa | Término de error | Incremento de sus pesos |
|---|---|---|
| Salida | $\delta_o = (y - \hat{y})\cdot\hat{y}(1 - \hat{y})$ | $\Delta w_ho = \eta\cdot\delta_o\cdot a_h$ |
| Oculta | $\delta_h = \delta_o\cdot w_ho\cdot a_h(1 - a_h)$ | $\Delta w_ih = \eta\cdot\delta_h\cdot x_i$ |

- a_h es la salida de la neurona oculta.
- La fórmula de la oculta es la misma regla delta, pero su "error" es **$\delta_o$ multiplicado por el peso que la une con la salida**.

Viene de la regla de la cadena (Clase 4): el error depende de w_ih a través de h_h, luego de a_h, luego de la salida. Solo hay un eslabón más.

### Ejemplo resuelto: pase hacia adelante (diapositiva 14)

Red 3 → 2 → 1 del notebook `1_10`:

- x = [0.5, 0.1, −0.2], objetivo = 0.6, η = 0.5;
- W_ih = [[0.5, −0.6], [0.1, −0.2], [0.1, 0.7]] y W_ho = [0.1, −0.3].

| Valor | Cálculo | Resultado |
|---|---|---|
| Entrada a la oculta | x·W_ih | [0.24, −0.46] |
| Salida oculta a_h | σ(…) | [0.5597, 0.3870] |
| Entrada a la salida | 0.5597·0.1 + 0.3870·(−0.3) | −0.0601 |
| Salida ŷ | σ(−0.0601) | **0.4850** |

### Ejemplo resuelto: hacia atrás (diapositiva 15)

| Valor | Cálculo | Resultado |
|---|---|---|
| Error | 0.6 − 0.4850 | 0.1150 |
| $\delta_o$ | 0.1150 · 0.4850 · 0.5150 | **0.02873** |
| δ_h | 0.02873 · [0.1, −0.3] · a_h(1 − a_h) | **[0.000708, −0.002045]** |
| Δw_ho | 0.5 · 0.02873 · [0.5597, 0.3870] | **[0.00804, 0.00556]** |
| Δw_ih | 0.5 · δ_h · x (matriz 3 × 2) | ≈ [[0.000177, −0.000511], …] |

Los incrementos tienen **el mismo tamaño** que los pesos: cada peso necesita su propio incremento.

### El gradiente que se desvanece (diapositiva 16)

Los incrementos de la capa oculta son **mucho más pequeños** que los de la salida. El error llega multiplicado por pesos pequeños y por derivadas de la sigmoide, que valen como mucho 0.25.

Con muchas capas, esto se repite y las primeras capas casi no aprenden. Es el problema del **gradiente que se desvanece**, y una razón por la que las redes profundas usan ReLU.

### El algoritmo completo (diapositiva 18)

Para cada época:

1. Poner en cero los incrementos acumulados.
2. Para cada ejemplo: pase hacia adelante → errores $\delta_o$ y δ_h → acumular los incrementos.
3. Actualizar las dos matrices de pesos con η · acumulado (o con el promedio).
4. Medir la pérdida.

---

# PARTE C — Práctica en la computadora

Para cada notebook, la guía detallada (celda por celda, con pistas y valores) está en `docs/guias_notebooks/`.

## C1. `notebooks/1_09_red_multicapa_ejercicio.ipynb` (20 min, 3 puntos) — diapositiva 10

- Tamaños de las capas: 4, 3 y 2.
- W_1 de (4, 3) y W_2 de (3, 2), con `np.random.normal`, media 0 y desviación 1/√4.
- Pase hacia adelante con `np.dot`. Ojo: la segunda capa usa **A_1**, no H_1.
- Celda extra con sesgos B_1 (1 × 3) y B_2 (1 × 2).

✅ **Comprobación:**

- la capa oculta da 3 números entre 0 y 1, y la salida 2;
- en total hay 23 parámetros;
- con la semilla 42 la salida es ≈ [0.3210, 0.3627].

Guía: NB_1_09.

## C2. `notebooks/1_10_retropropagacion_base.ipynb` (15 min, 3 puntos) — diapositiva 17

Los valores de B3 son exactamente los que deben salir con η = 0.5.

Error típico: en Δw_ih hay que convertir x en columna con `x[:, None]` para obtener la matriz 3 × 2.

Guía: NB_1_10.

## C3. `notebooks/1_11_retropropagacion_completo.ipynb` y XOR (20 min, 3 puntos) — diapositivas 18–19

Datos de admisiones (los de `1_07`), red 6 → 2 → 1, 900 épocas, semilla 21:

| Actualización | Pérdida | Exactitud |
|---|---|---|
| 0.001 · acumulado (plantilla) | 0.2646 → 0.2013 | 0.725 |
| 0.5 · acumulado / n_records | 0.2641 → 0.1999 | **0.750** |

⚠️ El notebook original imprime la pérdida usando solo el último ejemplo (`x`). Cámbialo por `features.values`.

**Cierre en vivo:**

```
python docs/practica_05.py --graficar
```

Una red 2 → 3 → 1 aprende XOR: predice 0.025, 0.981, 0.971 y 0.026, que redondeados dan 0, 1, 1 y 0. La frontera que aprendió **ya no es una recta**.

Guía: NB_1_11.

---

## Quiz y cierre (10 min) — diapositivas 20–21

| Pregunta | Respuesta |
|---|---|
| ¿Por qué una capa oculta permite resolver XOR? | Cada neurona oculta traza una recta y la salida las combina |
| ¿Qué tamaño tiene W entre una capa de 4 y una de 3? | 4 × 3 |
| ¿Cuántos parámetros tiene una red 2 → 3 → 1? | (2+1)·3 + (3+1)·1 = 13 |
| ¿Cómo se calcula el error de una neurona oculta? | $\delta_h = \delta_o \cdot$ peso hacia la salida · su derivada |
| ¿Qué es el gradiente que se desvanece? | El error se hace muy pequeño al ir hacia atrás por muchas capas |

**Para llevar:**

1. Las capas ocultas combinan rectas y resuelven problemas no lineales.
2. Pase hacia adelante = la Clase 4 repetida capa por capa.
3. Retropropagación = mandar el error hacia atrás, repartido según los pesos.

**Tarea:**

1. Terminar `1_09`, `1_10` y `1_11` (9 puntos).
2. En `docs/practica_05.py`, cambia `n_ocultas` a 2 y a 8: ¿sigue aprendiendo XOR? Cambia también la `semilla`.
3. Leer del libro: Retropropagación.

**Rúbrica:**

| Notebook | Puntos |
|---|---|
| 1_09 | 3 |
| 1_10 | 3 |
| 1_11 | 3 |

## PARTE D — Problemas frecuentes

| Síntoma | Causa | Solución |
|---|---|---|
| `ValueError: shapes ... not aligned` | Orden de las matrices o tamaños mal puestos | Sigue la regla (a × b)·(b × c); usa `.shape` para revisar |
| `operands could not be broadcast together` en Δw_ih | x no está como columna | `x[:, None]` |
| Los signos de los δ salen al revés | Usaste ŷ − y | Es **y − ŷ** |
| `1_11` no aprende (exactitud ≈ 0.42) | Dividiste entre n_records con una tasa muy pequeña (0.005) | Tasa 0.5 si divides, o 0.001 sin dividir |
| La pérdida impresa en `1_11` salta | El original usa solo el último ejemplo | Usa `features.values` |
| `practica_05.py` no aprende XOR con otra semilla | Algunas inicializaciones se atascan | Prueba otra semilla o más neuronas ocultas (pasa también en redes reales) |